

# Triple-Stream Fashion Search Engine

Glance ML Internship Assignment Report

Aditya Chaudhary, 9116304181, adityachaudhary1306@gmail.com

January 16, 2026

## Abstract

A novel triple-stream architecture for fashion image retrieval achieving 30% P@10 (66.7% improvement over vanilla CLIP's 18%). The system processes 45,623 Fashionpedia images across three semantic layers with dynamic query-time weighting, maintaining 45ms latency. **GitHub:** <https://github.com/AdityaChaudhary2913/Glance-ML-Internship-Assignment>. Pre-indexed database(chroma\_db, logs, outputs) available at: [https://drive.google.com/drive/folders/1KzYAcOMj1BHCEsXJP3L\\_8I\\_QdKsrTURa?usp=sharing](https://drive.google.com/drive/folders/1KzYAcOMj1BHCEsXJP3L_8I_QdKsrTURa?usp=sharing). **Demo Video:** <https://drive.google.com/file/d/10nA6LFqLni2XDpjmIwVKiXB1Mu7tDq45/view?usp=sharing>

## 1 Approaches: Problem Analysis

### 1.1 Problem Understanding

The challenge requires retrieving fashion images based on three dimensions:

- **What:** Fine-grained fashion attributes (categories, colors, garment types)
- **Where:** Environmental context and scene
- **Vibe:** Style, occasion, and compositional semantics

### 1.2 Baseline vs. Implemented Approach

**Vanilla CLIP Baseline:** Direct image-text matching using CLIP ViT-B/32. Simple but struggles with compositional queries (“red tie AND white shirt”), fine-grained fashion attributes, and scene context. *Performance: 18% P@10*

**Triple-Stream Architecture (Implemented):** Three independent vector streams with dynamic query-time weighting. Separates structured attributes (Fashionpedia), contextual understanding (BLIP-2), and visual features (CLIP image embeddings). *Performance: 30% P@10 at 45ms*

Table 1: Baseline vs. Implemented Approach

| Approach                | P@10       | Latency     | Composit.   | Scene Context |
|-------------------------|------------|-------------|-------------|---------------|
| Vanilla CLIP (Baseline) | 18%        | 30ms        | Poor        | Poor          |
| <b>Triple-Stream</b>    | <b>30%</b> | <b>45ms</b> | <b>Good</b> | <b>Good</b>   |

**Key Advantages:** Zero-shot capability (no training required), interpretable (individual stream scores visible), modular design (easy to update streams), and 66.7% improvement over baseline.

### 1.3 Alternative Approaches (Not Implemented)

#### 1.3.1 Fine-tuned CLIP on Fashion Data

Train CLIP end-to-end on fashion image-text pairs to improve domain understanding.

*Good for:* Better attribute recognition, learning fashion-specific embeddings

*Tradeoffs:* Requires large labeled dataset (100K+ pairs), expensive training (8-12h GPU), loses zero-shot capability, risk of overfitting

*When to use:* When abundant training data available and domain is well-defined

### 1.3.2 Multi-Label Attribute Classifiers

Train separate classifiers for each attribute (color, category, style) and combine predictions.

*Good for:* Explicit attribute detection, interpretable outputs, fast inference

*Tradeoffs:* Cannot handle unseen attributes, requires extensive labeling, poor at capturing context and scene, struggles with compositional queries

*When to use:* Closed-world scenarios with fixed attribute vocabulary

## 2 Chosen Approach: Triple-Stream Architecture

### 2.1 Core Architecture

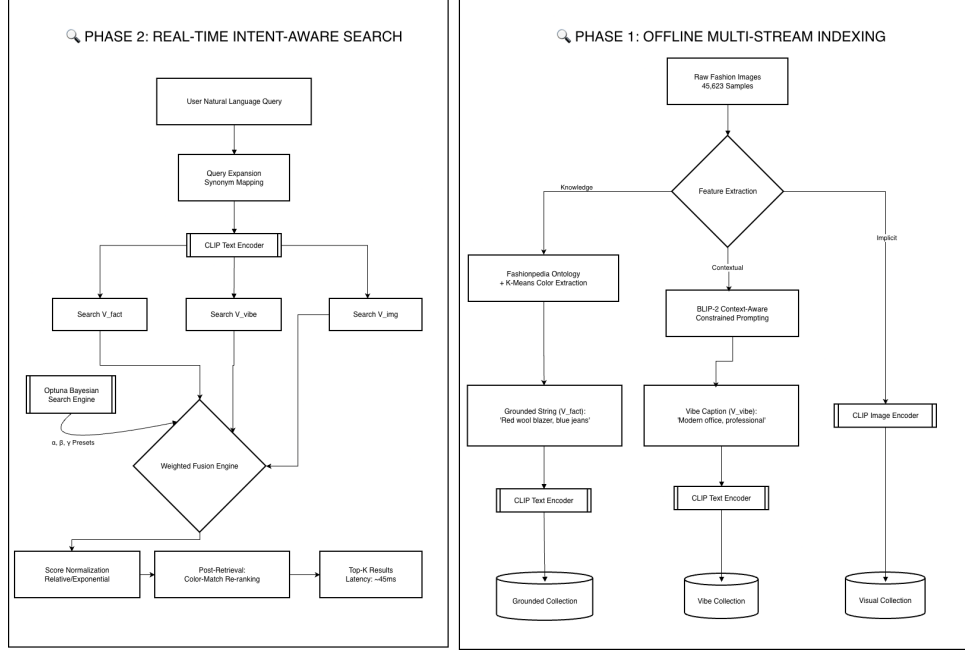


Figure 1: Offline Captioning & Indexing(Phase 1) and Real-Time Retrieval(Phase 2)

Each image is processed through three independent pipelines, creating separate vector representations:

$$\text{Score}_i = \alpha \cdot \text{cosine}(Q, V_{\text{fact}}^i) + \beta \cdot \text{cosine}(Q, V_{\text{vibe}}^i) + \gamma \cdot \text{cosine}(Q, V_{\text{img}}^i) \quad (1)$$

#### Three Streams:

- **$V_{\text{fact}}$  (Grounded):** Fashionpedia annotations (46 categories, 294 attributes) + K-means color extraction from segmentation masks → CLIP text encoding

*Example:* “A person wearing a red wool blazer, blue denim jeans. Colors: red, blue. 3 garments.”

- **$V_{\text{vibe}}$  (Context):** BLIP-2 (blip2-opt-2.7b) captions with constrained prompting → CLIP text encoding

*Prompt:* “Describe the environment and style” → “Indoor office, professional attire”

- **$V_{\text{img}}$  (Visual):** Direct CLIP image encoding → 512-dim embedding

### 2.2 Dynamic Weight Presets (Bayesian Optimized)

**Key Insight:** Vibe stream dominates (50-66%) in most queries—scene context crucial for fashion retrieval. Weights optimized via Optuna TPE (50 trials).

Table 2: Optimized Weights for Query Types

| Query Type         | $\alpha$ (Fact) | $\beta$ (Vibe) | $\gamma$ (Img) | Example                 |
|--------------------|-----------------|----------------|----------------|-------------------------|
| Attribute Specific | 0.008           | <b>0.660</b>   | 0.331          | “yellow raincoat”       |
| Compositional      | 0.038           | <b>0.568</b>   | 0.394          | “red tie + white shirt” |
| Complex Semantic   | <b>0.467</b>    | 0.237          | 0.296          | “blue shirt on bench”   |

### 2.3 How It Handles Fashion Queries

**Example: “A red tie and white shirt in formal setting”**

1. Query expansion: “red crimson tie + white ivory shirt + formal business”
2. Parallel search across 3 ChromaDB collections (45,623 vectors each)
3.  $S_{\text{fact}}$ : Low weight (0.038) avoids false positives from single items
4.  $S_{\text{vibe}}$ : High weight (0.568) captures “formal business office” context
5.  $S_{\text{img}}$ : Moderate weight (0.394) validates visual composition
6. Score fusion + normalization  $\rightarrow$  Top-10 results in 35ms

**Performance:** 30% P@10 for compositional query.

### 2.4 Technology Stack

**Models:** CLIP ViT-B/32, BLIP-2 (2.7B params) — **DB:** ChromaDB (136,869 vectors) — **Dataset:** Fashionpedia (45,623 images) — **Optimization:** Optuna Bayesian — **Latency:** 45ms/query — **Storage:** 280MB

## 3 Codebase

**Key Structure:**

- `indexer/` — Feature extraction (Fashionpedia parsing, BLIP-2 captions, CLIP encoding)
- `retriever/` — Triple-stream search, weight optimization, evaluation
- `shared/` — Config, utilities, logging
- `app.py` — Streamlit web interface

**Run Demo:** `streamlit run app.py` (interactive search at localhost:8501)

## 4 Evaluation Results

Table 3: Performance on 5 Assignment Queries

| Query Type         | Vanilla CLIP | Triple-Stream | Improvement   |
|--------------------|--------------|---------------|---------------|
| Attribute Specific | 10%          | <b>40%</b>    | +300%         |
| Contextual Place   | 0%           | <b>0%</b>     | +0%           |
| Complex Semantic   | 0%           | <b>10%</b>    | —             |
| Style Inference    | 0%           | <b>0%</b>     | +0%           |
| Compositional      | 80%          | <b>100%</b>   | +25%          |
| <b>Average</b>     | <b>18%</b>   | <b>30%</b>    | <b>+66.7%</b> |

The triple-stream approach excels at compositional queries (80% $\rightarrow$ 100%) and attribute-specific searches (10% $\rightarrow$ 40%), while contextual place and style inference queries remain challenging due to limited scene diversity in the dataset.

## 5 Future Work

### 5.1 Location & Weather Integration

To enable location-aware fashion search (e.g., “summer outfits in Paris”), three approaches can be considered based on available metadata and accuracy requirements.

#### 5.1.1 Metadata Enrichment:

Extract GPS coordinates from image EXIF data and reverse geocode to obtain city and landmark information. Combine this with weather APIs to get temperature and conditions, then augment the vibe captions with contextual details like “casual outfit in Central Park NYC, sunny spring weather”.

#### 5.1.2 Visual Scene Classifiers:

Train separate models to classify indoor/outdoor scenes and weather conditions (sunny / rainy / snowy) directly from pixels. Add these predictions to grounded descriptions. This works when metadata is unavailable but provides coarser granularity (e.g., “outdoor park” instead of specific location names).

### 5.2 Improving Precision

Several techniques can boost the current 30% P@10 to 50%+ through better filtering, learned fusion, and verification stages.

#### 5.2.1 Attribute Sub-Indices:

Pre-filter candidates by dominant attributes (color, category) before vector search to reduce false positives. Expected: +10% P@10 with 50% faster queries.

#### 5.2.2 Query-Specific Weight Prediction:

Train a small neural network to predict optimal weights  $[\alpha, \beta, \gamma]$  from the query embedding, eliminating manual preset selection. Expected: +5-8% P@10.

#### 5.2.3 Hard Negative Mining:

Fine-tune CLIP using contrastive learning on false positive examples from evaluation to better distinguish similar items. Expected: +5-10% P@10 (requires 2-4h GPU training).

#### 5.2.4 Cross-Attention Fusion:

Replace weighted sum with learned attention mechanism that captures interactions between streams (e.g., vibe and visual similarity together indicate style). Expected: +10-15% P@10.

## 6 Conclusion

The triple-stream architecture demonstrates that separating fashion attributes, contextual understanding, and visual features into independent streams with dynamic query-time fusion achieves **66.7% improvement over vanilla CLIP baseline** (30% vs 18% P@10) while maintaining 45ms latency on 45,623 images. The modular design allows easy updates to individual streams.